FairShare

Архитектура и рабочий MVP

Студенческая командная работа: понятные задачи, посильная нагрузка, видимые препятствия.

Проект по M1 Product Vision · 2 октября 2026

01 Архитектура

02 Функции группы

03 Алгоритм распределения

04 Данные и права

05 Развитие и гипотезы

06 Источники

07 Визуальная спецификация

Исходный код, API-контракт и SQL-схема входят в комплект. Реализованные возможности отделены от будущих; результаты пользовательского исследования не выдумываются.

Архитектура FairShare

Проектирование: 2 октября 2026 года. Основание — заполненный M1, страницы 2–4. Документ описывает реализованный MVP и обозначает дальнейшее развитие отдельно.

1. Продукт и ограничения

FairShare помогает студентам распределить учебный этап на понятные задачи, согласовать нагрузку и своевременно показать препятствия. Основная пользовательская цепочка: **этап → задачи с контекстом → доступность и навыки → рекомендация → обсуждение и подтверждение → работа → результат**.

В PDF наблюдения основаны на прошлых проектах самой команды, а принятие алгоритма и точность оценки часов остаются гипотезами. Нельзя считать часы или число действий объективным вкладом. Отказ от публичных рейтингов «пассивности», автоматических оценок и обвинений — продуктовый выбор, а не ограничение базы.

Рабочее допущение для первой версии: небольшие студенческие команды, несколько параллельных проектов, один недельный интервал планирования. Ограничения масштаба ещё не подтверждены нагрузочным исследованием.

2. Какой стек лучше именно здесь

Вариант	Сильные стороны	Ограничения	Решение
React + Vite + отдельный API	Удобен для досок, форм и совместного рабочего пространства; понятные границы клиента и сервера	Нужно самостоятельно организовать сессии и API	Выбран
Next.js	SSR, публичные страницы, серверный рендеринг, экосистема React	Для закрытой доски SSR не даёт существенной продуктовой выгоды; добавляет второй серверный механизм	Хорошая альтернатива при появлении публичного портала
Vue / Svelte	Подходят для того же продукта	Переход не улучшает основной сценарий, а пользователь предпочёл React	Не нужны для этой версии
Flutter / React Native	Нативные мобильные сценарии, push	Установка приложения и отдельное тестирование платформ увеличивают стоимость первого эксперимента	После подтверждения мобильной потребности
Electron / Tauri	Настольное приложение	Нет потребности в локальных файлах, IDE или системном API	Не требуется

Вариант	Сильные стороны	Ограничения	Решение
Firebase / Firestore	Быстрый старт Auth и синхронизации	Связи команды, задачи, зависимости, версии и атомарные назначения удобнее моделировать реляционно	PostgreSQL лучше соответствует данным

Официальная документация React рекомендует начинать с фреймворка. Здесь осознанно выбран SPA на Vite: учебный продукт имеет отдельный API, закрытую рабочую область, не требует SEO и позволяет проверить основной сценарий без SSR. Это решение для FairShare, не универсальное утверждение о React.

Конкретные технологии

Слой	Реализовано	Причина
Клиент	React, TypeScript, Vite, CSS tokens, Inter, Lucide	Интерактивная доска, повторно используемые компоненты, читаемые типы
HTTP API	Node.js, Fastify, Zod	Небольшой сервер; валидация на границе; явные feature-модули
Домен	Чистая TypeScript-функция распределения	Можно проверять отдельно от HTTP и базы
Данные	PostgreSQL: PGlite локально, адаптер pg для внешней БД	Один язык SQL и транзакционные сценарии, без установки Docker для локального MVP
Идентификация	Email/пароль, scrypt, случайные серверные сессии	Рабочий локальный вход без внешнего аккаунта и обязательных ключей
Синхронизация	REST + опрос каждые 15 секунд	Простая согласованная первая версия; пользовательские изменения сразу перечитываются
Проверка	node:test, Fastify inject, Playwright Chromium	Проверки домена, API и реального интерфейса

Точные установленные версии зафиксированы package-lock.json. Управляемая PostgreSQL допустима у любого совместимого провайдера. Supabase изучен как вариант PostgreSQL/Auth; текущий код не использует Supabase SDK и не требует Supabase-проекта. Если выбрать его базу, приложение подключается серверным адаптером pg; Data API для этих таблиц должен быть закрыт. Подключение Supabase Auth — отдельная будущая замена модуля Identity, а не реализованная функция.

3. Архитектурный стиль

Модульный монолит: один серверный процесс, одна реляционная база, несколько функциональных модулей. Он подходит команде из 2–3 разработчиков: распределённые транзакции, отдельные очереди и согласование контрактов микросервисов пока не дают

полезного результата.

Использованы принцип разделения ответственности, типизированные границы, композиция UI, независимый доменный алгоритм, общий порт базы Sql/Database, транзакционные сценарии, optimistic locking для задач и явная проверка полномочий. Называть MVP строгой реализацией Clean Architecture было бы неточно: SQL и сервисные сценарии находятся рядом с HTTP-обработчиками внутри feature-модулей. При росте их можно вынести в application/ и repositories/, сохранив HTTP-контракт и доменную функцию.

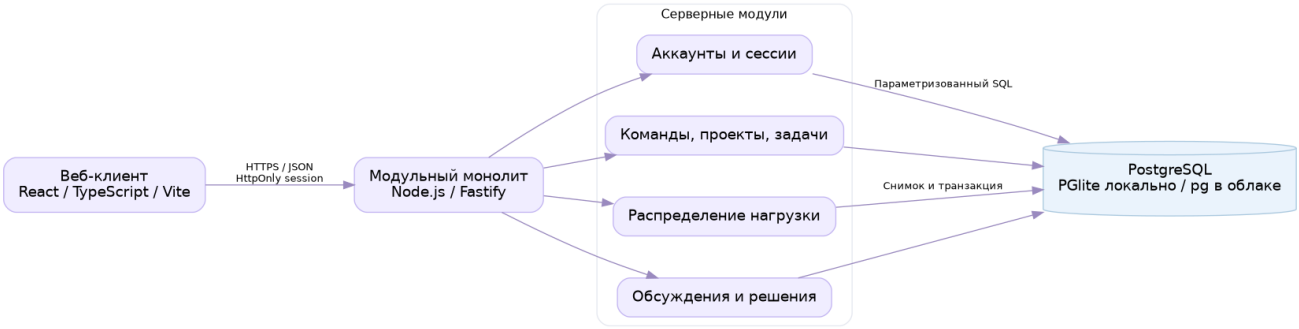
Контекст системы

Схема в Mermaid; исходник доступен в соответствующем Markdown-документе.

Внешние документы хранятся у владельцев; FairShare сохраняет только ссылку и название. Приложение не загружает код и не анализирует его качество.

Контейнеры и данные

Схема в Mermaid; исходник доступен в соответствующем Markdown-документе.



Это C4-подобные схемы в Mermaid, а не формальный UML. Статическая SVG-версия контейнеров предназначена для чтения вне Markdown. Текст вокруг схемы — её доступная альтернатива. Широкие SVG следует прокручивать, а не уменьшать до нечитаемого размера.

4. Модули и владение

Модуль	Ответственность	Основные файлы
Identity	Аккаунты, пароль, сессии, демовход	server/auth.ts, server/security.ts
Workspace	Доступ к проекту, список участников, агрегированная нагрузка	server/workspace.ts
Projects	Команды, проекты, этапы, приглашения, доступность	server/projects.ts
Tasks	Контекст задачи, статусы, зависимости, помощь, результат	server/tasks.ts
Allocation	Предложение и транзакционное принятие	server/allocation.ts, server/allocations.ts

Модуль	Ответственность	Основные файлы
Collaboration	Обсуждения, шаги, материалы, решения и голоса	server/collaboration.ts
Persistence	Общий SQL-порт, адаптеры, схема, учебный seed	server/db.ts, server/schema.sql, server/seed.ts
Web	Навигация, рабочие представления, диалоги	src/components, src/features
Contracts	Типы клиента/сервера и OpenAPI	shared/types.ts, contracts/openapi.yaml

В новой большой версии сервисный сценарий должен зависеть от порта репозитория, а не импортировать Fastify. Домен распределения уже соблюдает эту границу: не знает о cookies, HTTP, PostgreSQL и React.

5. Сценарий подтверждения распределения

Схема в Mermaid; исходник доступен в соответствующем Markdown-документе.

Членство, доступность и уже назначенная работа — общий контекст нескольких проектов. Записи команды и пользователей блокируются до изменения; пользователи — в стабильном порядке UUID, чтобы уменьшить риск взаимных блокировок. Доступность пользователя блокирует его запись. Подтверждение повторно читает снимок под блокировками. Защита работает на сервере, независимо от состояния кнопки клиента.

Снимок включает версию этапа, IDs и версии задач, состав команды, навыки, часы доступности и нагрузку. Изменение задачи в другом проекте за ту же неделю изменит нагрузку, поэтому старое предложение также отклоняется. Повторное принятие уже принятого предложения возвращает успех без повторного назначения: идемпотентность.

6. Клиентская архитектура

App координирует сессию, выбор проекта и этапа; AppShell отвечает за навигацию. Доска, нагрузка, команда, личные задачи и журнал — отдельные feature-компоненты. Общие Modal, Button, Avatar, Progress задают единое поведение.

API-клиент использует same-origin cookies. При сохранении данные перечитываются с сервера, что исключает расхождение оптимистичного интерфейса с проверкой прав. Номер запроса защищает основное пространство от устаревшего ответа при переключении проектов. В будущей версии с большим числом экранов нужен TanStack Query: ключи вида [projectId, milestoneId], отмена запросов, дедупликация и централизованная инвалидизация.

Перетаскивание не требуется для управления задачей: статус меняется в доступной форме. Клавиатура, фокус в диалогах, Escape, мобильная компоновка, контраст текста и reduced motion поддерживаются. Это набор реализованных мер; полную сертификацию WCAG без отдельного аудита заявлять нельзя.

7. Размещение и развитие

Локально один Node-процесс раздаёт сборку React и API; PGlite хранит данные на диске. PGlite не следует подключать одновременно к нескольким процессам или держать на временном диске hosting-платформы.

Для пилота: статическая сборка + Node API за HTTPS, управляемая PostgreSQL, секреты окружения, отдельные окружения demo/staging/production. Можно оставить React и API на одном домене: меньше сложностей с cookies и CORS. Провайдер размещения не связан с доменом продукта.

При горизонтальном масштабировании сначала используются обычная PostgreSQL, распределённый rate limit и общие правила сессий. Для push-синхронизации — SSE/WebSocket с проверкой членства и устойчивым курсором событий. Если появятся письма/уведомления, событие и запись outbox сохраняются одной транзакцией, worker доставляет их с повторными попытками. Redis нужен при подтверждённой нагрузке очередей или presence; Kubernetes не является условием запуска.

8. Нефункциональные цели и честные границы

Цели пилота, ещё не подтверждённые нагрузочным измерением: p95 обычного API до 500 мс при 50 активных пользователях; распределение 100 задач между 10 участниками до 1 секунды; синхронизация чужих изменений до 15 секунд плюс сетевое время; восстановление production-базы за 4 часа, допустимая потеря до 24 часов при ежедневной резервной копии.

Реализованы транзакционное принятие, контроль версий, проверки членства, сессии и история. Не реализованы восстановление пароля, проверка email, файловое хранилище, email/push, полноценные versioned migrations для уже существующей production-базы, индивидуальное принятие назначения каждым участником и multi-instance инфраструктура. Эти работы описаны в дорожной карте, а не скрыты за макетом.

Функции для работы в группе

«Работает» означает реализацию в API и интерфейсе. Остальные возможности — каталог развития, а не обещание уже готового кода. Основание — M1 и дополнительный запрос на расширение группового функционала.

Работает в MVP

Функция	Польза для команды
Регистрация, вход, выход и сохраняемая сессия	Личные права и договорённости
Своя команда, несколько команд у пользователя	Разные учебные группы в одном приложении
Одноразовое приглашение на 24 часа	Самостоятельное подключение участников
Несколько проектов и недельных этапов	Разделение предметов и промежуточных результатов
Контекст задачи: цель, первый шаг, результат	Понятный старт без постоянного объяснения координатора
Оценка часов и обязательный навык	Обсуждаемый объём и посильность работы
Личная доступность по неделям	Учёт занятий и внешних обязательств
Навыки, которые указывает сам участник	Прозрачные ограничения назначения
Общая нагрузка во всех проектах	Одни свободные часы не расходуются несколько раз
Объяснимое предложение распределения	Исходный вариант для обсуждения
Подтверждение координатором после обсуждения	Решение остаётся за командой
Защита от устаревшего предложения	Новые задачи и доступность не игнорируются
Ручное назначение с подтверждением исключения	Гибкость с видимым риском перегрузки
Доска трёх состояний	Общая картина выполнения
Личные задачи во всех проектах	Понятные личные обязательства
Зависимость задачи и защита от циклов	Явный порядок работы
«Нужна помощь» в один клик и пояснение	Ранняя видимость препятствия
Ссылка или описание результата	Сохранённый итог работы
Обсуждения в контексте задачи	Уточнения не теряются в общем чате
Общие чек-листы конкретных шагов	Задачи проще начинать и передавать
Общие ссылки на документы и репозиторий	Материалы доступны из проекта

Функция	Польза для команды
Предложения командных решений	Спорные вопросы формулируются явно
Голосование за/против, замена своего голоса	Один актуальный голос каждого участника
Фиксация решения координатором	Сохранение итога договорённости
Журнал последних действий	Контекст изменений без рейтингов людей
Обновление каждые 15 секунд и при возврате в окно	Видимость изменений коллег
Адаптивный интерфейс и клавиатурные диалоги	Работа с телефона и без drag-and-drop

Приоритет P1: ежедневная координация

Возможность	Сценарий	Архитектурное дополнение
Индивидуальное принятие назначения	«Подходит» / «Нужны изменения»	assignment_responses и правило активации плана
Передача задачи с контекстом	Участник просит замену и передаёт уже сделанное	handoff_requests, согласие принимающего
Парная работа	Новичок выполняет задачу с наставником	task_collaborators, один ответственный
Взаимная проверка	Автор передаёт результат рецензенту	review_requests, состояние in_review
Критерии приёмки	Команда заранее договаривается о готовности	Обязательные пункты acceptance checklist
Диапазон и уверенность оценки	Учёт риска недооценки	estimate_low/high, сценарии нагрузки
Совместная оценка	Независимые оценки участников	estimate_votes, медиана и разброс
Резерв часов	Запас на ошибки и обучение	capacity_reserve
Несколько зависимостей и граф	Видимость цепочек результата	task_dependencies, проверка DAG
Шаблоны этапов	Повторные лабораторные и презентации	Версионируемые templates
Шаблоны стартового контекста	Примеры начала UI, API, тестов, отчётов	Первый шаг, файлы, пример результата
Поиск и фильтры	По человеку, статусу, навыку, сроку	Пагинация, серверные фильтры, full-text
Архив семестра	Законченная работа не мешает текущей	archived_at и политика истории

Возможность	Сценарий	Архитектурное дополнение
Редактирование проектов и этапов	Уточнение названия и сроков	Контроль версии и пересчёт нагрузки
Упоминания	Адресная просьба помощи	Проверка членства, ограничение частоты
Подписка на задачу	Следить за нужной зависимостью	task_subscriptions
Центр уведомлений	Назначения, ответы и запросы помощи	notifications, read cursor, дедупликация
Личные напоминания	Участник сам выбирает сроки	notification_preferences, outbox, worker
Командная сводка	Один digest вместо множества сообщений	Планировщик и выбор каналов

Приоритет P2: договорённости и обучение

Возможность	Польза	Условия
Повестка встречи	Вопросы готовы до обсуждения	Ссылки на этап и задачи
Итоги встречи	Разговор превращается в действия	Решения отдельно от новых задач
Опрос времени встречи	Выбрать удобный слот	Часовые пояса и несколько вариантов
Асинхронный check-in	«Сделал / дальше / мешает»	Коротко и добровольно, без табеля часов
Доска рисков	Проблема видна до дедлайна	Эффект, вероятность, владелец реакции
Критический путь	Видны задерживающие цепочки	Реальные зависимости и длительности
Ретроспектива	Сохранение полезного опыта	Один согласованный эксперимент на следующий этап
База знаний	Примеры и решения остаются команде	Поиск и ответственный за актуальность
Соглашение команды	Общие ожидания по ответам и помощи	Версии и принятие участниками
Ротация учебных ролей	Координация и развитие распределяются	Учебная роль отдельно от системных прав
Личные учебные цели	Посильная задача для нового навыка	«Хочу изучить» отдельно от «умею»
Приватное пояснение доступности	Не нужно раскрывать причины всей команде	Для планирования достаточно итоговых часов

Возможность	Польза	Условия
История передачи и переоценки	Договорённости можно восстановить	Автор, причина, подтверждение
Экспорт проекта	Командный результат можно передать	Только разрешённые данные
Read-only наставник	Показать статус преподавателю	Ограниченные права и отзыв приглашения

Приоритет P3: интеграции

- GitHub/GitLab: прикрепление результата и проверки PR; коммиты не считаются качеством или вкладом.

- Файлы: приватное object storage, лимиты, сканирование, signed URLs.

- Общий календарь дедлайнов, ICS и calendar sync; синхронизация исключена из первой версии M1.

- Telegram/email digest: только по явной подписке участника на канал.

- PWA, offline-чтение и затем конфликтобезопасные offline-изменения.

- AI-черновик разбивки задания с проверкой человеком; AI syllabus parsing исключён из первой версии M1.

- Обезличенные продуктовые метрики для проверки гипотез, без публичного рейтинга студентов.

- CP-SAT/целочисленный оптимизатор, когда жадная эвристика на реальных данных окажется недостаточной.

Граница продукта

Полный мессенджер, IDE, автоматическое оценивание кода, обязательный ежедневный timesheet, публичный рейтинг «пассивности» и выставление учебных оценок не входят в FairShare. Контекстные обсуждения помогают выполнять задачи; основной чат может остаться в привычном инструменте.

Рекомендуемый порядок после MVP: индивидуальное согласие → передача → взаимная проверка → уведомления → шаблоны. Сложные интеграции добавляются после наблюдаемой потребности, чтобы приложение не увеличивало административную нагрузку.

Объяснимое распределение нагрузки

Данные и определение справедливости

Планируется одна неделя, начинающаяся в понедельник UTC. capacity(member, week) — заявленные участником часы для всех проектов за эту неделю. Не число часов в каждом отдельном проекте. Отсутствующая декларация считается нулём: приложение не придумывает свободное время.

load(member, week) — сумма оценок всех назначенных, незавершённых задач участника в этапах этой недели. Число завершённых задач, просмотры и комментарии не используются как оценка вклада. Заблокированная задача продолжает занимать плановые часы, пока команда не перенесёт или не изменит её.

Коэффициент загрузки: r = planned_hours / declared_hours. Например, 6 из 6 часов — полная загрузка; 6 из 12 — половина. Равные абсолютные часы или количество карточек дали бы другое, менее подходящее распределение.

Алгоритм реализованного MVP

1. Берём только planned задачи выбранного этапа. Начатые задачи сохраняют исполнителя. Завершённые не участвуют.

2. Из общей текущей нагрузки вычитаем назначения этих же запланированных задач, чтобы пересчитываемый этап не учитывался дважды.

3. Для каждой задачи определяем допустимых участников: доступность больше нуля и наличие обязательного навыка. Для «Любой навык» проверка навыка не нужна.

4. Сортируем задачи: сначала меньше подходящих участников, затем больше часов, затем UUID. Это защищает задачи с редким навыком от раннего расходования нужной ёмкости.

5. Исключаем участников, для которых задача целиком приведёт к превышению доступности. Частично назначать неделимую задачу алгоритм не умеет.

6. Выбираем минимальное прогнозируемое отношение (existing_load + task_estimate) / capacity. При равенстве — стабильный UUID.

7. Если участника нет, оставляем задачу без назначения и объясняем: нет навыка или нет свободных часов.

8. Показываем будущую нагрузку и причины назначений. Записываем предложение и снимок входных данных. До подтверждения реальные назначения не меняются.

9. Координатор подтверждает рекомендацию после командного обсуждения. Это не индивидуальные цифровые согласия всех участников; такой механизм предусмотрен следующей версией.

Функция находится в server/allocation.ts, не зависит от базы и HTTP. Сложность текущей простой реализации — приблизительно O(T × M log M + T log T) с дополнительными линейными проходами; для малых команд её достаточно.

Пример

У Ерасыла 12 часов и навыки Backend/Архитектура, у Суусар 8 часов и Frontend/Тестирование, у Сезим 6 часов и Документация/Базы данных. Уже в работе: интерфейс 5 часов у Суусар, презентация 2 часа у Сезим. Архитектура 4 часа и API 6 часов подходят Ерасылу, тесты 3 часа — Суусар.

Участник	Уже в работе	Предложенные задачи	Итог	Доступность
Ерасыл	0	4 + 6	10	12
Суусар	5	3	8	8
Сезим	2	0	2	6

Часы распределены не поровну, потому что навыки ограничивают допустимые назначения. Приложение не обязано искусственно назначать Сезим техническую задачу, которую она не заявляла как посильную.

Исключения и неопределённость

- При нехватке навыков предложить парную работу, обучение или разбивку задачи — полезная следующая функция. MVP показывает проблему и оставляет решение команде.

- При перегрузке уже принятой работой алгоритм не скрывает её и не добавляет новые задачи сверх часов.

- Ручное назначение может нарушить ограничения только после явного подтверждения координатором; факт исключения записывается в журнал.

- Оценка часов ненадёжна. В следующей версии нужны интервалы, уверенность, командная проверка оценок и резерв. Не следует автоматически считать заявленные часы истинным временем выполнения.

- Зависимость определяет порядок начала/завершения, но не точное расписание внутри недели. Алгоритм не проверяет критический путь и календарную выполнимость цепочки.

- Жадная эвристика может оставить задачу без участника, хотя другой порядок назначений дал бы допустимый план. Для более крупных команд можно сравнить её с integer programming или CP-SAT; оптимизатор пока не реализован.

Согласованность

Все изменения назначений принимаются одной транзакцией. Внешняя PostgreSQL использует блокировки команды и записей участников; задачи в UI дополнительно имеют version. Подпись предложения включает задачи, нагрузки, часы и навыки. Несовпадение возвращает HTTP 409. Повторное подтверждение принятого предложения ничего не переназначает.

Тестируются нулевая доступность, отсутствие навыка, нагрузка других проектов, сохранение начатой работы, отсутствие двойного счёта, редкие навыки, большие неделимые задачи, разные отношения часов к доступности и повторяемость результата.

Данные, права и эксплуатация

Сущности

Таблица	Назначение	Ограничения
users	Аккаунты, навыки	Уникальный email, scrypt-хэш
sessions	Серверные сессии	Только хэш токена, срок, FK пользователя
teams / memberships	Команды и роли	PK team/user, один координатор на команду
availability	Часы на неделю	PK user/week, 0–100 часов
projects / milestones	Проекты и недельные этапы	FK команды и проекта, revision этапа
tasks	Контекст и ход работы	Составные FK этап/проект и зависимость/проект, часы > 0, version
allocations	Предложения распределения	Снимок, автор, payload, состояние принятия
invitations	Одноразовые приглашения	Хэш токена, срок, used_by
events	История действий	Проект, пользователь, тип, минимум контекста
comments / checklist	Обсуждения и конкретные шаги	FK задачи, удаление вместе с ней
resources	Внешние материалы	Название, HTTP/HTTPS URL, автор
decisions / votes	Договорённости	Один голос на user/decision, явное закрытие

Схема в Mermaid; исходник доступен в соответствующем Markdown-документе.

Исполняемая схема: server/schema.sql. API дополнительно проверяет календарные даты, начало недели в понедельник, срок внутри недели, шаг оценки 0,25 часа, цикл зависимости, членство исполнителя и условия статуса. Прямое администраторское изменение БД обходит правила, не продублированные в SQL.

Состояния задачи

planned → in_progress → done. Можно вернуть задачу в план или сразу завершить её, если есть исполнитель, завершена зависимость и описан результат. Для завершения нужен текст/ссылка от 10 символов и снятый запрос помощи. Повторное открытие завершённой задачи требует проверки доступности. Help — отдельное поле, которое не скрывает основной статус.

started_at означает заявленное начало, first_evidence_at — первое описание/ссылку, done_at — завершение. Это технические события, которые не доказывают содержательность работы

автоматически.

Права

Действие	Координатор	Участник
Читать свою команду	Да	Да
Читать чужую команду	Нет	Нет
Создать задачу	Да	Да
Менять условия и назначение	Да	Нет
Менять статус, помощь, результат	Любая задача команды	Только собственная назначенная
Удалять актуальную planned-задачу без зависимых	Да	Нет
Менять собственные навыки/часы	Да	Да
Менять чужие часы	Нет	Нет
Создавать проект, этап, приглашение	Да	Нет
Предложить и принять распределение	Да	Нет
Обсуждать, добавлять шаги и материалы	Да	Да
Предлагать решение, голосовать	Да	Да
Фиксировать итог решения	Да	Нет

Сервер проверяет членство на каждом запросе. UUID не даёт доступ. Объект вне своей команды возвращает 404; запрещённое действие — 403. Отдельного суперпользователя в пользовательском API нет.

Реализованная защита

- Пароли: индивидуальная соль и scrypt. В БД и логах нет открытых паролей.

- Сессия: 32 случайных байта, в БД SHA-256-хэш, срок 7 дней. Cookie HttpOnly/SameSite=Strict, в production Secure. Выход удаляет серверную сессию.

- Проверка Origin и Sec-Fetch-Site на изменяющих запросах; в production точный APP_ORIGIN. Произвольный CORS не включён.

- Zod, параметризованный SQL, тело до 100 KB. Вход/регистрация ограничены 12 запросами в минуту на IP текущего процесса.

- React экранирует текст. Общие ссылки принимают HTTP/HTTPS, открываются с noopener/noreferrer.

- API: no-store. Статические ответы: CSP, nosniff, no-referrer, DENY iframe.

- Приглашения одноразовые, 24 часа, хранятся хэши. Отправка остаётся действием пользователя.

- Задачи имеют version; принятие распределения атомарно и проверяет подпись снимка. Блокируются команда и участники в стабильном порядке.

- NODE_ENV=production отключает демовход по умолчанию. Отключение маршрута не удаляет учебные аккаунты: внешнему пилоту нужна отдельная чистая база.

Перед внешним пилотом

Проверка email, восстановление пароля, политика хранения/удаления, резервные копии с проверенным восстановлением, наблюдения ошибок, versioned migrations и разные учётные данные runtime/migration пока не реализованы. В текущем MVP схема идемпотентно создаёт таблицы; это не полноценный механизм эволюции production-базы.

Если выбрать Supabase-провайдер: закрыть Data API для серверных таблиц и grants клиентских ролей или использовать неэкспонируемую схему. Любые экспонируемые таблицы требуют RLS; service/secret key нельзя помещать в React. В текущем приложении данные защищаются API-проверками, клиент не имеет доступа к БД. Supabase Auth не подключён.

Несколько API-процессов требуют обычной PostgreSQL, общего rate limit и проверки конкурентных блокировок. PGlite предназначена для одного процесса; нельзя одновременно открывать один её каталог двумя серверами или хранить его на временном диске hosting-платформы.

Хранение и эксплуатация

В MVP нет удаления аккаунта/проекта, архива и удаления обсуждения. Просроченные сессии проверяются на каждом запросе и очищаются при старте. Добавление управления сроками хранения входит в дорожную карту.

/api/health проверяет БД. CI: npm ci → typecheck → test → build. Локальную PGlite резервировать после корректного завершения процесса; перенос в обычную PostgreSQL требует экспорта/импорта. Production использует резервные копии PostgreSQL-провайдера и регулярную проверку восстановления в отдельное окружение. Предлагаемые RPO 24 часа и RTO 4 часа — цели пилота, не измеренный SLA.

Этапы развития и проверка продукта

Уже реализовано

Рабочий локальный MVP: идентификация и команды, проекты и недельные этапы, контекст задач, доступность/навыки, рекомендации и атомарное подтверждение, доска/помощь/результаты, обсуждения/чек-листы, материалы и командные решения. Код разделён по модулям; схема PostgreSQL, адаптеры, типы и проверки находятся в репозитории.

Следующие этапы

Этап	Результат	Критерий готовности
Пилот 1, 1 неделя	2 команды проходят основной сценарий	Измерены понятность первого шага, начало работы, время настройки
Пилот 2, 1–2 недели	Проверена полезность рекомендаций	Зафиксированы принятие, причины ручных изменений и ошибки оценок
Согласование, 1–2 недели	Личное принятие назначения и передача	Участник явно соглашается с изменением обязательства
Взаимная проверка, 1–2 недели	Автор передаёт результат рецензенту	Замечания, исправление и принятие сохраняются рядом с задачей
Эксплуатация, 1–2 недели	Готовность внешнего ограниченного пилота	Email verification/reset, migrations, резервирование и наблюдения проверены
Удобство, 1–2 недели	Шаблоны, поиск и уведомления	Сокращается ручная работа без обязательных ежедневных форм

Сроки — предварительная оценка для команды из 2–3 человек, не обязательство. Работы и исследование могут идти частично одновременно. Интеграции с календарём и AI остаются поздними, как предусмотрено M1.

Проверка гипотез M1

H1: понятный контекст помогает начать работу

Участники: две реальные студенческие команды, 4–6 студентов, добровольное участие. Базовый период — сопоставимая задача в привычном чате; затем трёхдневный этап из шести структурированных задач. Команда читает цель, первый шаг и ожидаемый результат до начала.

«Начало содержательной работы» — первый содержательный документ, черновик или кодовый коммит в пределах 48 часов, проверенный исследователем. Перевод карточки в «В работе» сам по себе этим доказательством не является. first_evidence_at помогает найти момент прикрепления, но содержательность и фактическое время артефакта проверяются отдельно.

Измерения: время до первого артефакта, число вопросов «с чего начать», необходимость вмешательства координатора, способность участника своими словами назвать первый шаг и результат. Условия поддержки гипотезы из PDF: участники начинают в пределах 48 часов, вопросов заметно меньше, ожидания понятны. Дополнительный порог можно согласовать заранее, например относительное снижение вопросов на 30%; он ещё не проверен и не является фактом.

Если участники продолжают бездействовать, исследовать отдельно навыки, расписание, зависимости и мотивацию. Приложение не может принудить к усилию.

H2: рекомендации принимаются как исходная договорённость

Записать число показанных и принятых предложений, ручные изменения и их объяснения. Спросить участников, какие ограничения алгоритм учёл и чего не учёл. Одно нажатие координатора не доказывает согласие остальных: нужно короткое интервью команды.

H3: оценок часов достаточно для планирования

Сравнить исходную оценку с добровольной итоговой оценкой усилия и качественными причинами отклонения. Не вводить обязательный ежедневный timesheet. При систематической недооценке добавить диапазоны, резерв и взаимную проверку оценок. История done_at - started_at означает календарное время, не количество фактически отработанных часов.

Снижение административной нагрузки

Измерить время подключения первого участника и создания первой понятной задачи; цель M1 для первоначального onboarding — меньше 3 минут. Необходимо отдельно измерить полный путь настройки проекта: три минуты не являются обещанием всей настройки системы.

Метрики продукта

Метрика	Определение	Что она не доказывает
Понятность старта	Доля участников, верно объяснивших первый шаг и результат	Мотивацию и выполнение
Начало в 48 часов	Доля задач с вручную проверенным первым артефактом	Качество финального результата
Принятие рекомендаций	Доля принятых предложений и причины правок	Согласие всей команды без интервью
Своевременная помощь	Время между известным препятствием и сигналом помощи	Отсутствие незафиксированных препятствий
Перегрузка плана	Часы назначений сверх объявленной доступности	Фактическую перегрузку вне приложения
Пользовательское трение	Время настройки и повторных действий	Рыночный спрос

Нельзя придумывать участникам отзывы, результаты тестов или значения метрик. В проекте пока есть учебные данные и техническая верификация; пользовательское исследование не проводилось.

Рекомендованное распределение работы команды

Один участник ведёт UI и доступность, второй — API/данные/домен, третий — тестирование и исследование, документацию и приёмку. При двух участниках третья роль разделяется. Области ответственности не должны препятствовать совместному review: каждый должен уметь объяснить продукт, модель нагрузки и основные проверки.

Интернет-источники и решения

Проверены 2 октября 2026 года. В research/sources.json сохранены адреса, HTTP-статусы и время получения, в research/*.txt — полученный текст. Это проверка официальной документации, а не пользовательское исследование.

Источник	Что проверено	Влияние на FairShare
React: Creating a React App	React рекомендует фреймворки, допускает самостоятельную сборку при соответствующих ограничениях	SPA/Vite выбран осознанно для закрытой доски с отдельным API; Next.js остаётся альтернативой
Vite: Guide	TypeScript/React шаблоны, требования к Node	Воспроизводимая сборка и актуальный Node
PostgreSQL: Explicit Locking	Row-level locks, FOR UPDATE, взаимные блокировки	Блокировка команды и пользователей в стабильном порядке
PostgreSQL: Transaction Isolation	Согласованность конкурентных транзакций	Повторное чтение снимка и атомарное принятие
OWASP: Authorization Cheat Sheet	Deny by default, полномочия на каждом запросе	Членство в команде проверяется сервером, а не только UI
TanStack Query: Overview	Управление server state, кеширование и синхронизация	Рекомендован для дальнейшего роста; в MVP не подключён
WCAG 2.2	Клавиатура, фокус, доступность и адаптивность	Focus trap, видимый фокус, мобильная компоновка; без заявления о сертификации
Supabase: Securing your API	Экспонирование схем, grants и Data API	Если выбран как PostgreSQL-провайдер, серверные таблицы не должны открываться браузеру
Supabase: RLS	Политики доступа к строкам	RLS нужен для экспонируемых таблиц; отдельный API не включает его автоматически
Supabase: JWTs	Верификация токенов и signing keys	Будущая замена Identity требует настоящей проверки JWT; текущий MVP использует серверные сессии
Supabase: Changelog	Актуальные изменения, breaking changes	Не предполагается, что облачный API остаётся неизменным

Индекс Supabase сообщает изменение PostgreSQL minor versions от 25 сентября 2026 года с предупреждением о переиндексации отдельных extensions и legacy-cipher данных. MVP не использует ltree, btree_gist, pgcrypto-шифрование или существующую Supabase-базу; конкретную миграцию чужого проекта не выполняли. Документация signing keys также проверена через официальный поиск Supabase MCP.

Страница NestJS была получена, но содержала только короткую JavaScript-заглушку. Она не использована как подтверждение архитектурного решения. Выбор Fastify связан с размером

feature-модулей и простотой небольшого монолита; это инженерное решение, не результат сравнительного бенчмарка.

Требования продуктовой границы и гипотез взяты из пользовательского PDF, а не из интернет-источников. Ссылка на Gemini в PDF не проверялась; приведённые там выводы учтены как записанные решения команды, а не новая беседа с Gemini.

Визуальная спецификация FairShare

Источник: diagrams/ui-concept.png, 1536 × 1024. Это рабочий экран приложения: открытая доска, три колонки, отдельная правая область нагрузки. Маркетинговой страницы внутри рабочего пространства нет.

Токены и геометрия

- Основной фон: холодный почти белый #f7f8fb; поверхности и боковая панель: #ffffff.

- Основной текст: #171928; вторичный: #737d96; границы: #e5e8f0.

- Акцент: #7056e8; светлая подложка: #eeebff; помощь: #fff5df, текст #946018.

- Шрифт: Inter при наличии, иначе системный sans-serif; отдельные размеры для заголовков, кнопок, подписей и полей.

- Заголовок этапа: 38–40 px, вес 700; интерфейс 14–16 px; подписи 12–14 px.

- Панель навигации: 278 px на широком экране, 230 px на ноутбуке; верхняя полоса 64 px; отступы основного пространства 30 px.

- Радиус карточки 12 px, кнопки 9 px. Тень едва заметная. Без градиентов и декоративных иллюстраций.

- Иконки Lucide: 18–22 px, толщина 1.7–2; знак равенства — смысловой UI-значок, бренд остаётся текстом.

Разрешённый состав первого экрана

FairShare; Работа в равновесии; Мои проекты; Мои задачи; Как это работает; ПРОЕКТЫ; Инженерия ПО; Базы данных; Пригласить; ISE4133 · Team FairShare; Первый рабочий прототип; Понятные задачи. Посильная нагрузка. Общий результат.; Срок этапа; динамическая дата; количество задач, часов и участников; Начните с честного распределения; Сравните нагрузку с доступным временем каждого.; Распределить задачи; Доска; Нагрузка; Команда; Новая задача; Запланировано; В работе; Готово; Нагрузка команды; План на эту неделю; Помощь нужна вовремя; Считаем запланированные часы, а не вклад или оценки.; Откройте задачу, чтобы увидеть первый шаг и ожидаемый результат.

Данные задач и имена берутся из API. Счётчики, дата и нагрузка меняются вместе с данными: макет не является источником бизнес-данных.

Дополнительные обязательные состояния

- Вход и регистрация; отдельно обозначенный деморежим.

- Диалог распределения с объяснениями и явным подтверждением.

- Диалог задачи: цель, первый шаг, результат, оценка часов, навык, исполнитель, зависимость, состояние, запрос помощи, ссылка или описание результата.

- Создание проекта и этапа, выбор этапа при нескольких этапах.

- Команда: редактирование собственной доступности и навыков, создание одноразового приглашения.

- Нагрузка: таблица часов и отношения к доступности.

- Обработка загрузки, пустого проекта, отсутствия подходящих участников, устаревшего предложения, ошибок доступа.

Это функционально необходимые продолжения макета. На мобильном экране навигация скрывается за кнопкой меню, доска становится вертикальной; нагрузка располагается ниже. Диалоги доступны с клавиатуры: удерживают фокус, закрываются Escape и возвращают фокус инициатору. Видимый focus-ring обязателен. Уменьшенная анимация учитывает prefers-reduced-motion.